

Stop Guessing, Start Understanding: How Arrow and Pandas Exchange Data

...

Alenka Frim and Raúl Cumplido

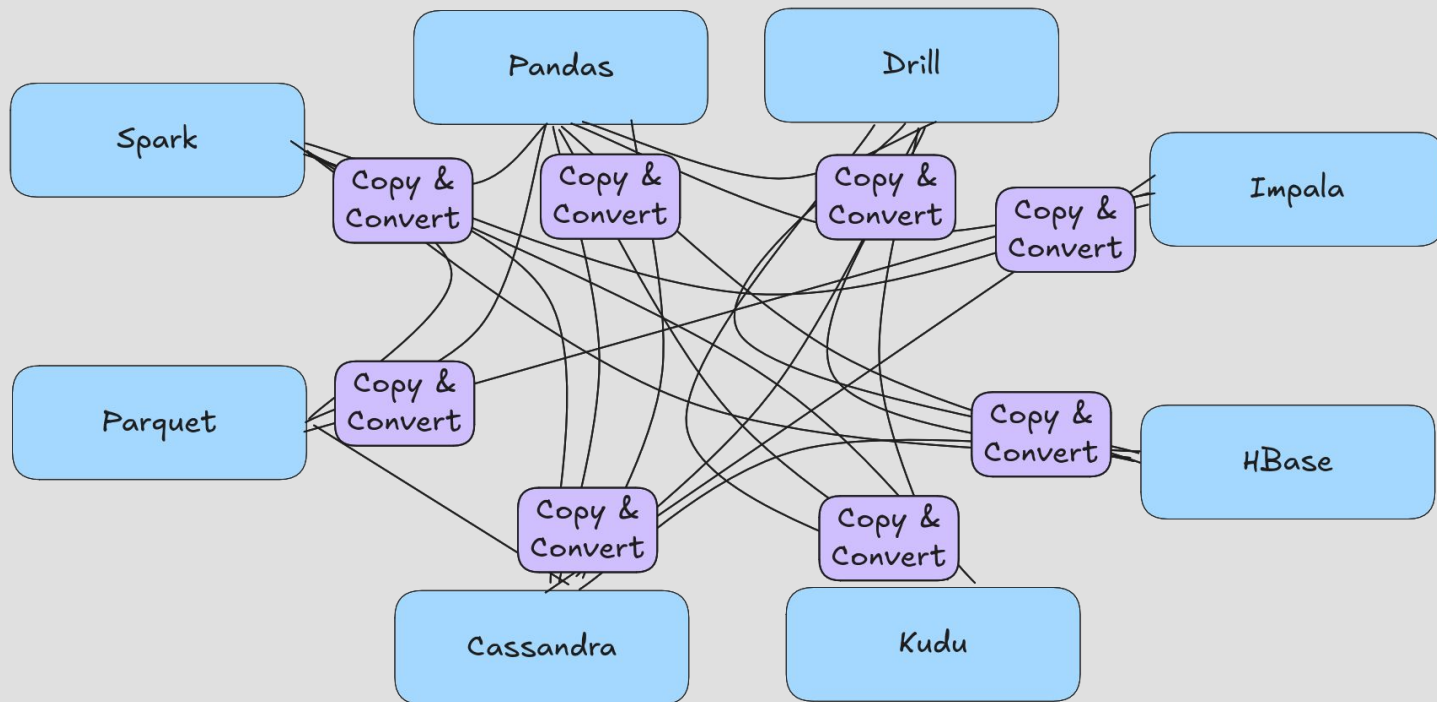
Two Libraries
Two Jobs

Are they competing?

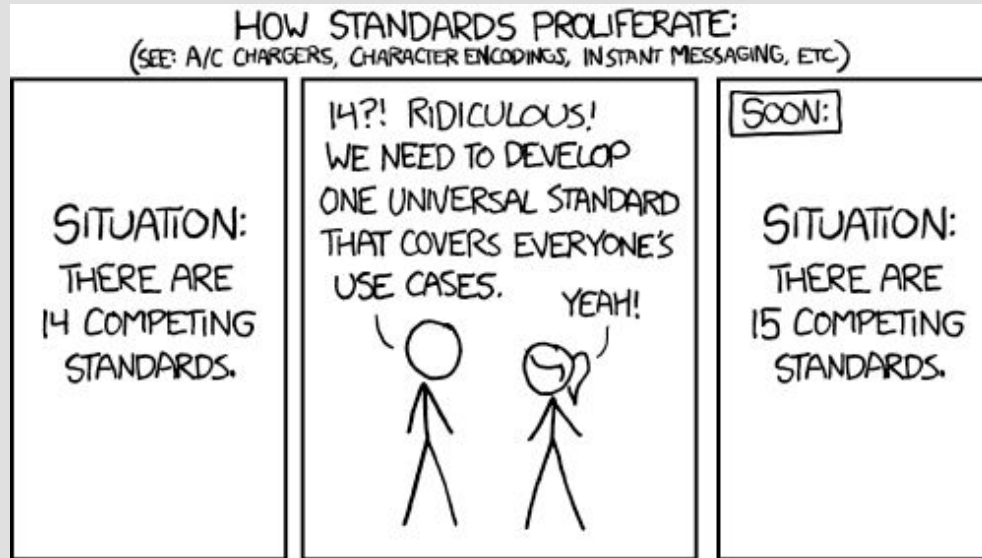


Background Screenshot from "Street Fighter 2"

The initial problem

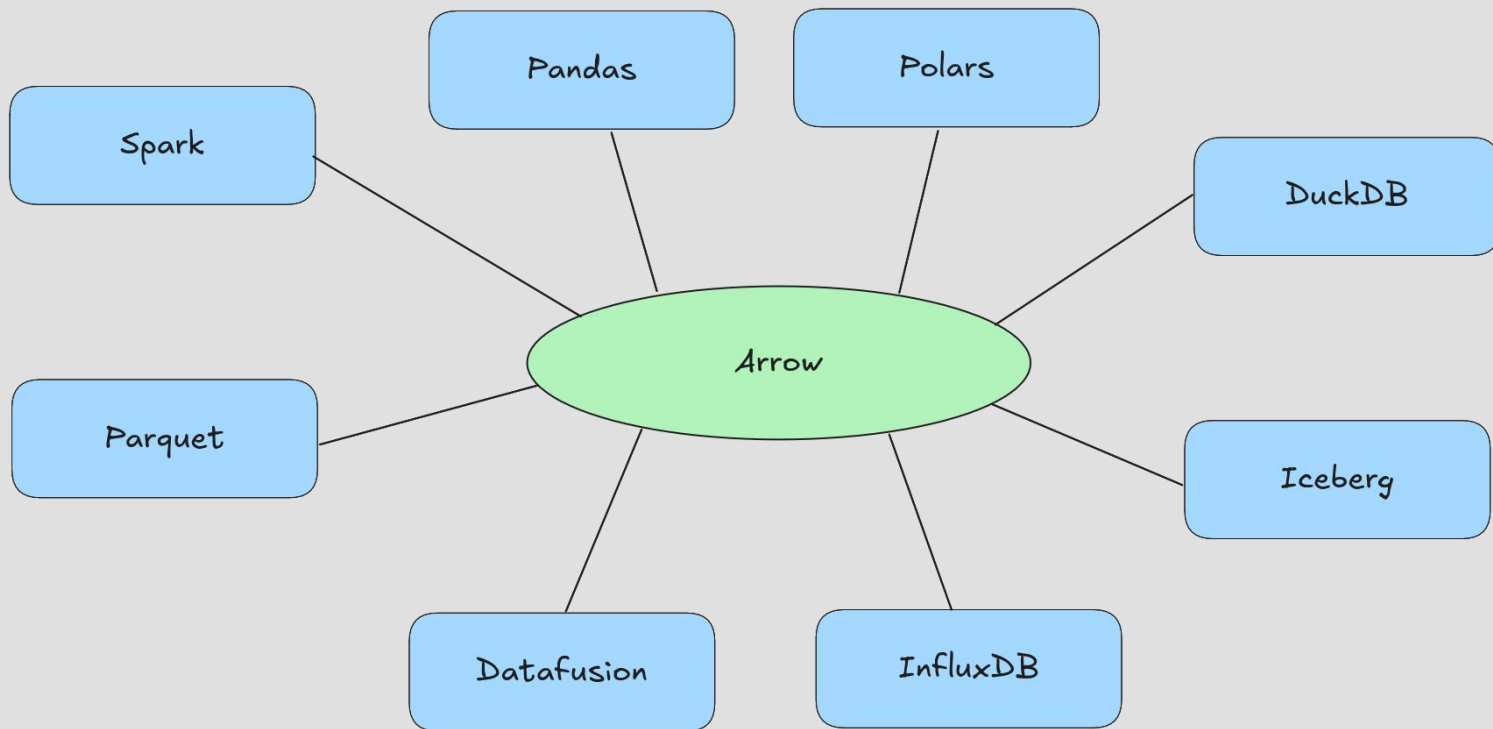


The initial idea



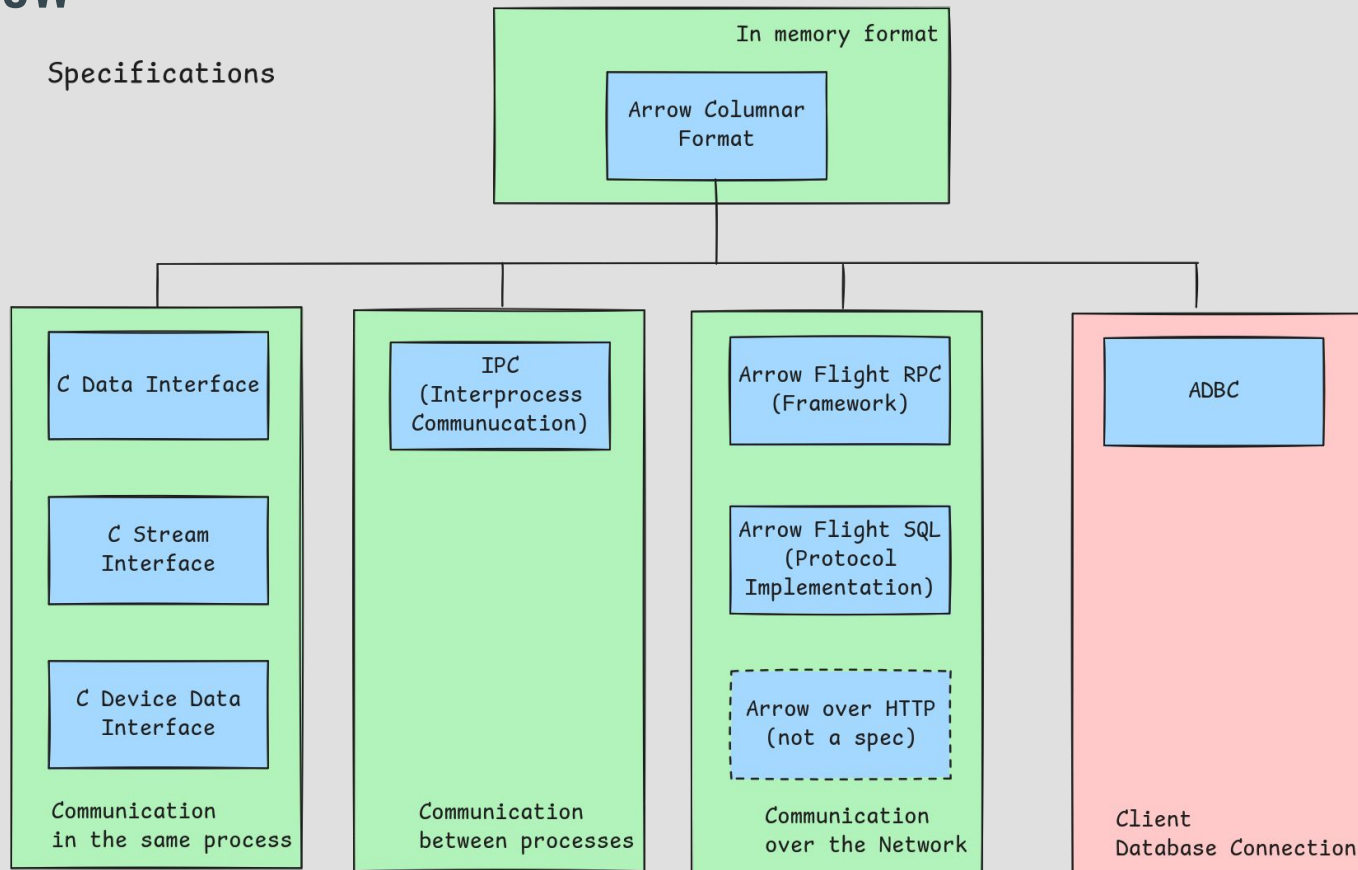
<https://xkcd.com/927/>

Arrow as a solution



Overview

Specifications



They are collaborating!

Memory format
Speed interop

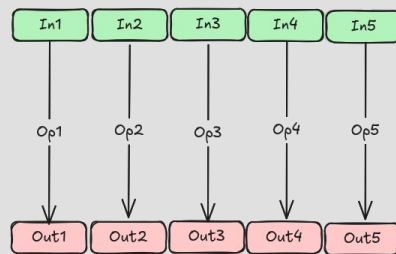


Powerful API
Data Analysis

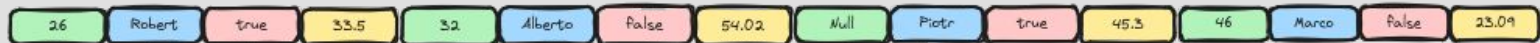
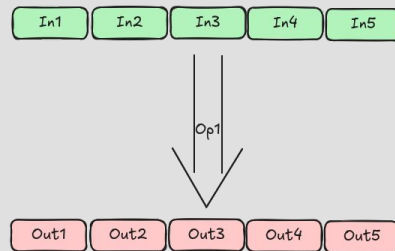
Memory Models

Age	Name	subscribed	spent
26	Robert	true	33.5
32	Alberto	false	54.02
Null	Piotr	true	45.3
46	Marco	false	23.09

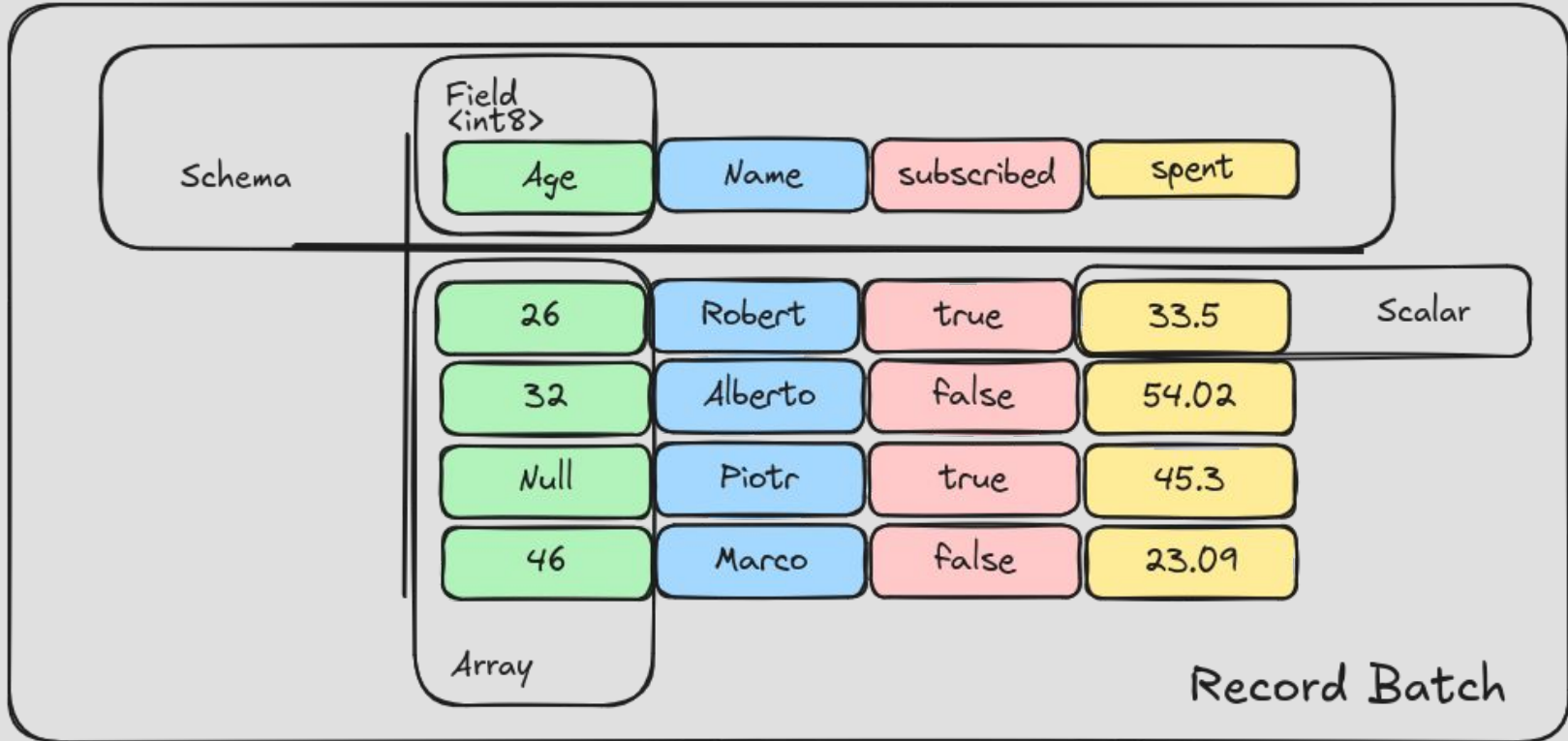
SISD (Single Instruction Single Data)



SIMD (Single Instruction Multiple Data)



Record batch and Array



pyarrow Table

id <int64>	Name <string>	Country <string>	Cost <int64>
1	Robert	UK	550
2	Alberto	Portugal	540
3	Piotr	Poland	200
4	Marco	Italy	690

Buffers

id: Int64Array

Data

1 2 3 4

Validity

None

Name: StringArray

Data

'RobertAlbertoPiotrMarco'

Offsets

0 6 13 18 23

Validity

None

Country: StringArray

Data

'UKPortugalPolandItaly'

Offsets

0 2 10 16 21

Validity

None

id: Int64Array

Data

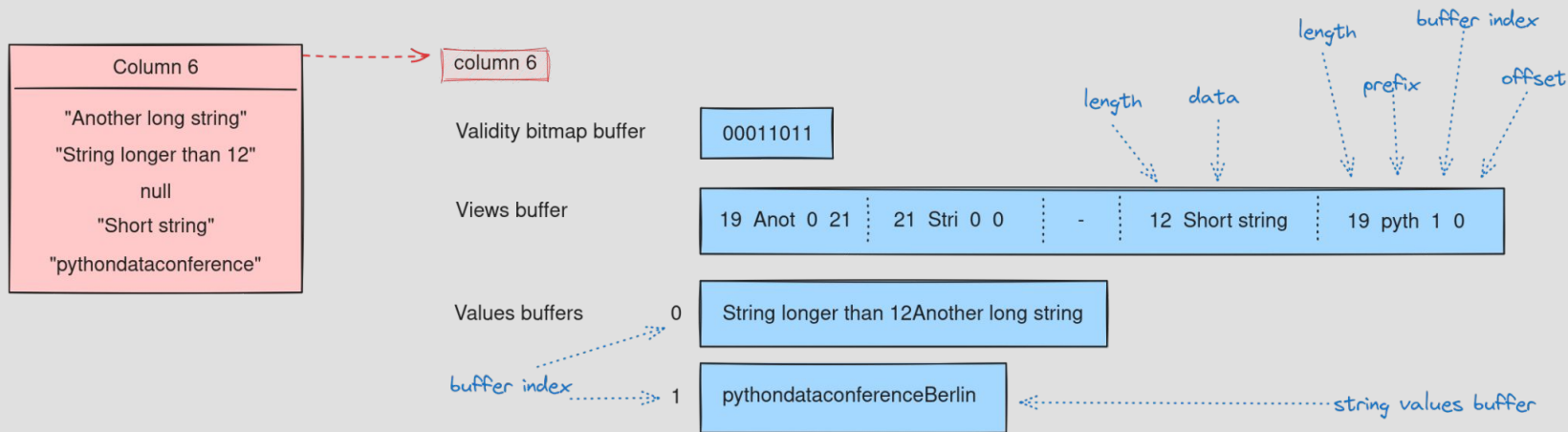
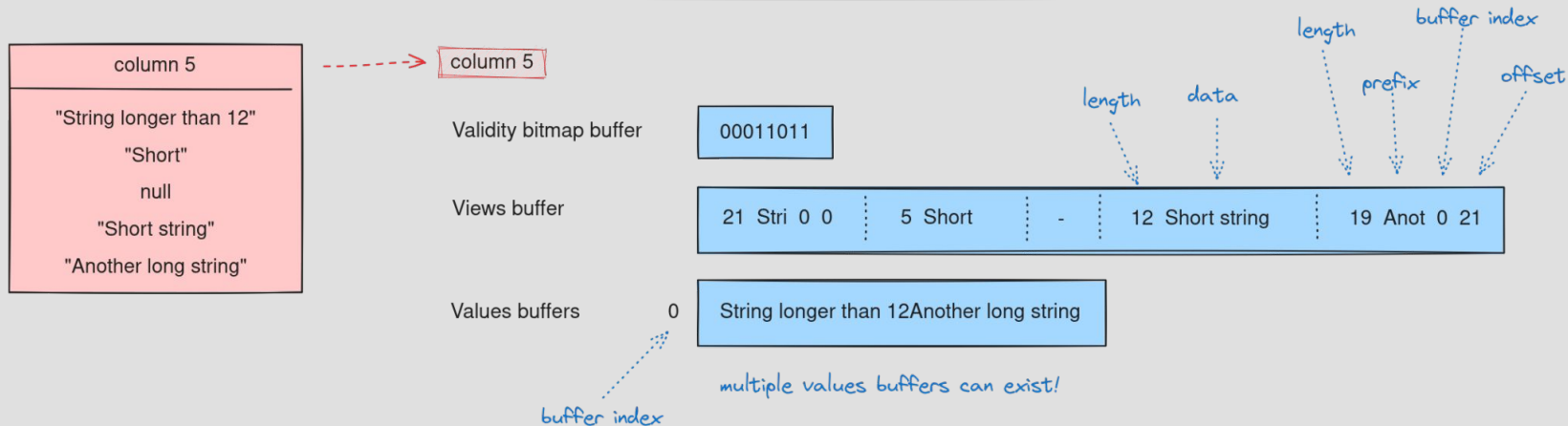
550 540 200 690

Validity

None

string view

Physical layout - Variable-size Binary View Layout



pandas DataFrame with object string dtype

id <int64>	Name <object>	Country <object>	Cost <int64>
1	Robert	UK	550
2	Alberto	Portugal	540
3	Piotr	Poland	200
4	Marco	Italy	690

BlockManager

NumpyBlock
dtype: int64 (numpy.ndarray)

[[1, 2, 3, 4],
[550, 540, 200, 690]]

C_CONTIGUOUS

NumpyBlock
dtype: object (numpy.ndarray)

[[ptr1, ptr2, ptr3, ptr4],
[ptr5, ptr6, ptr7, ptr8]]

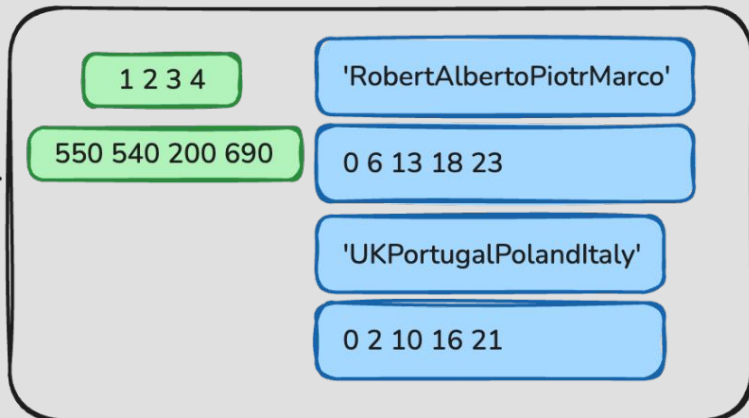
C_CONTIGUOUS

Python str objects

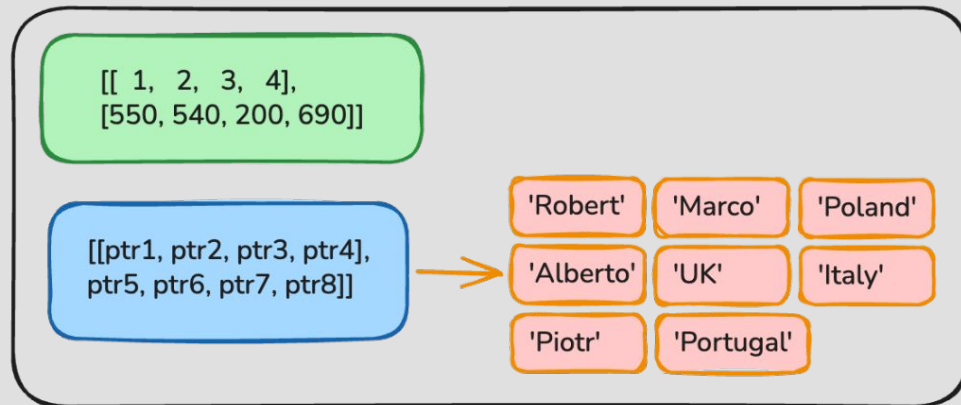
'Robert' 'Marco' 'Poland'
'Alberto' 'UK' 'Italy'
'Piotr' 'Portugal'

id <int64>	Name <string>	Country <string>	Cost <int64>
1	Robert	UK	550
2	Alberto	Portugal	540
3	Piotr	Poland	200
4	Marco	Italy	690

PyArrow



Pandas



pandas DataFrame with new string dtype

id <int64>	Name <str>	Country <str>	Cost <int64>
1	Robert	UK	550
2	Alberto	Portugal	540
3	Piotr	Poland	200
4	Marco	Italy	690

BlockManager

NumpyBlock
dtype: int64 (numpy.ndarray)

[[1, 2, 3, 4],
[550, 540, 200, 690]]

C_CONTIGUOUS

ExtensionBlock
dtype: str (ArrowStringArray)

'RobertAlbertoPiotrMarco'

0 6 13 18 23

None

ExtensionBlock
dtype: str (ArrowStringArray)

'UKPortugalPolandItaly'

0 2 10 16 21

None

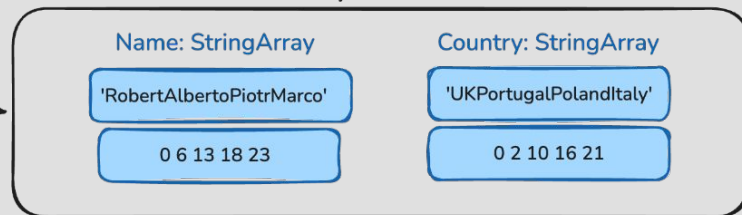
The Conversion

Pandas - NumPy backed string columns

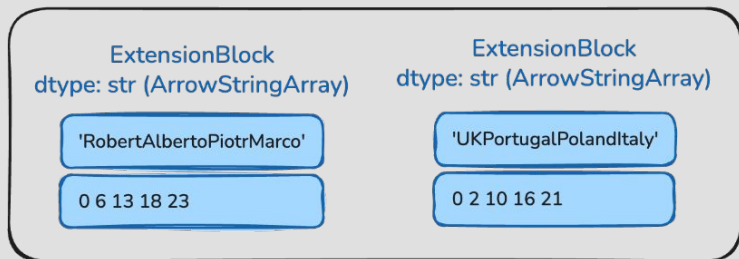


PyArrow
compat

PyArrow

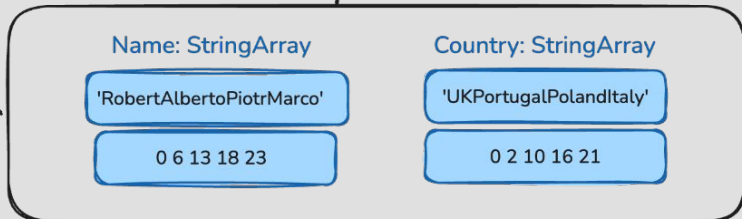


Pandas - PyArrow backed string columns



same
object

PyArrow



one-dimensional NumPy array with NaN

NumpyBlock
numpy.ndarray

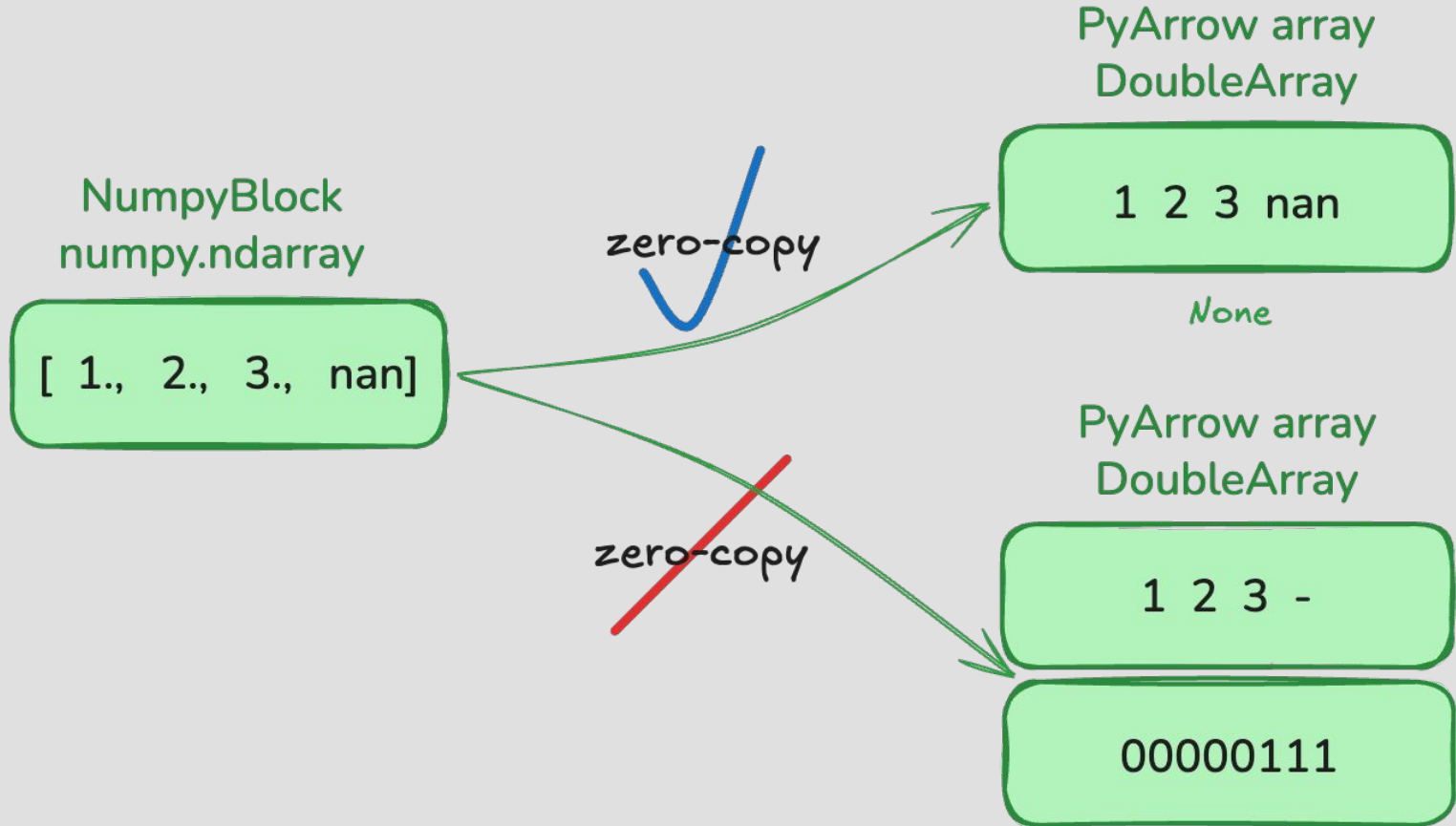
[1., 2., 3., nan]



PyArrow array
DoubleArray

1 2 3 nan

one-dimensional NumPy array with NaN



multidimensional NumPy array

NumpyBlock
numpy.ndarray

```
[[ 1, 2, 3, 4],  
 [550, 540, 200, 690]]
```



PyArrow arrays
2xInt64Array

```
1 2 3 4
```

```
550 540 200 690
```

multidimensional NumPy array

NumpyBlock
numpy.ndarray

[[1, 2, 3, 4],
[550, 540, 200, 690]]



zero-copy:

- not strided
- not byte-swapped

PyArrow arrays
2xInt64Array

1 2 3 4

550 540 200 690

one-dimensional IntegerArray with pd.NaN

ExtensionBlock
dtype: Int64 (IntegerArray)

[1, 2, 3, pd.NAN]



2D numpy.ndarray

[1, 2, 3, 1]

[False, False, False, True]



PyArrow array
Int64Array

1 2 3 -

00000111

one-dimensional IntegerArray with pd.NaN

ExtensionBlock
dtype: Int64 (IntegerArray)

[1, 2, 3, pd.NAN]



2xnumpy.ndarray

[1, 2, 3, 1]

[False, False, False, True]



PyArrow array
Int64Array

1 2 3 -

00000111

NumpyBlock
dtype: object (numpy.ndarray)

[[ptr1, ptr2, ptr3, ptr4],
ptr5, ptr6, ptr7, ptr8]]



Python str objects

'Robert'

'Marco'

'Poland'

'Alberto'

'UK'

'Italy'

'Piotr'

'Portugal'



PyArrow arrays
2xStringArray

'RobertAlbertoPiotrMarco'

0 6 13 18 23

'UKPortugalPolandItaly'

0 2 10 16 21

one-dimensional NumPy boolean array

NumpyBlock
numpy.ndarray

[True, False, True]



PyArrow array
BooleanArray

10100000

Practical Implications

Interoperability

Nullable

Numpy Compute

Mutation

Strings

Third-party libraries



serialization engine: pyarrow

serialization engine: fastparquet

Intermediate representation:
PyArrow Table

dtype_backend: pyarrow

dtype_backend: numpy_nullable

default

BlockManager

NumpyBlock
dtype: int64 (numpy.ndarray)

[[1, 2, 3, 4],
[550, 540, 200, 690]]

C_CONTIGUOUS

NumpyBlock
dtype: object (numpy.ndarray)

[[['Robert', 'Alberto', 'Piotr', 'Marco'],
['UK', 'Portugal', 'Poland', 'Italy']]]

C_CONTIGUOUS

ExtensionBlock
dtype: int64[pyarrow]

1 2 3 4

None

ExtensionBlock
dtype: large_string[pyarrow]

'RobertAlbertoPiotrMarco'

0 6 13 18 23

None

ExtensionBlock
dtype: large_string[pyarrow]

'UKPortugalPolandItaly'

0 2 10 16 21

None

ExtensionBlock
dtype: int64[pyarrow]

550 540 200 690

None

ExtensionBlock
dtype: Int64 (IntegerArray)

[1, 2, 3, 4]

[False, False, False, False]

ExtensionBlock
dtype: large_string[pyarrow]

'RobertAlbertoPiotrMarco'

0 6 13 18 23

None

ExtensionBlock
dtype: large_string[pyarrow]

'UKPortugalPolandItaly'

0 2 10 16 21

None

ExtensionBlock
dtype: Int64 (IntegerArray)

[550, 540, 200, 690]

[False, False, False, False]

BlockManager

NumpyBlock
dtype: int64 (numpy.ndarray)

[[1, 2, 3, 4],
[550, 540, 200, 690]]

C_CONTIGUOUS

ExtensionBlock
dtype: str (ArrowStringArray)

'RobertAlbertoPiotrMarco'

0 6 13 18 23

None

ExtensionBlock
dtype: str (ArrowStringArray)

'UKPortugalPolandItaly'

0 2 10 16 21

None

BlockManager

BlockManager

Takeaways

